

Sistema RAG Local con LM Studio y LangChain

Práctica Final de Curso

Asignatura: Sistemas Aprendizaje Automático + Modelos IA

Titulación: Inteligencia Artificial y Big Data

Alumno: Daniel Moro Fernández

Fecha: Abril 2026

Proyecto ejecutado al 100% en local - Sin APIs de pago ni envío de datos a la nube

1. Descripción del sistema

He desarrollado un sistema RAG (Retrieval-Augmented Generation) completamente local que permite a los empleados de una empresa consultar sus manuales internos a través de un chat con inteligencia artificial, sin que ningún dato salga de su ordenador. El sistema lee documentos en formato PDF o TXT, los trocea, los almacena en una base de datos vectorial y, cuando el usuario hace una pregunta, recupera los fragmentos más relevantes para que un modelo de lenguaje genere una respuesta fundamentada exclusivamente en esos documentos.

El aspecto que más distingue a mi implementación del ejemplo base del enunciado es que he diseñado una plataforma multi-proyecto. Esto quiere decir que desde la interfaz web se pueden crear varios espacios de trabajo independientes (los he llamado "Proyectos"), cada uno con sus propios documentos, su propia base de datos vectorial y su propio historial de conversaciones. Así, si subo las políticas de RRHH en un proyecto y un manual técnico en otro, los datos nunca se mezclan entre sí.

Documentos utilizados

Para simular el escenario de la empresa he creado cuatro documentos de texto que recogen políticas internas ficticias de una compañía llamada Acme Corp:

- Política vacaciones.txt - Normativa de días laborables, devengo mensual y bonificaciones por antigüedad.
- Política teletrabajo.txt - Reglas del trabajo remoto, equipamiento subvencionable, sanciones por incumplimiento.
- Ciberseguridad.txt - Normas de uso de dispositivos, contraseñas, VPN y servicios en la nube.
- Política gastos de viaje.txt - Dietas, límites de alojamiento, transporte y justificación de facturas.

En total suman más de 21.000 caracteres de contenido, lo cual supera el mínimo de 10 páginas equivalentes que pide el enunciado.

2. Arquitectura implementada

El sistema sigue la arquitectura de cinco capas indicada en el enunciado, pero he añadido varios componentes extra que mejoran la experiencia del usuario y la calidad de las respuestas. A continuación describo cada capa tal y como la he implementado:

Capa 1 - Ingesta de documentos (ingest_service.py)

Cuando el usuario sube un archivo desde la interfaz web, el backend lo guarda en la carpeta exclusiva del proyecto activo y dispara el pipeline de ingesta. Este pipeline utiliza PyPDFLoader para PDFs y TextLoader para TXTs. A continuación, un RecursiveCharacterTextSplitter divide el contenido en fragmentos (chunks) de 500 caracteres con un solapamiento de 50. Elegí este tamaño porque tras probarlo con valores de 300 y 800, observé que 500 ofrece el mejor equilibrio: los fragmentos son lo bastante largos para mantener el contexto de un párrafo completo, pero lo bastante cortos para que el modelo no se pierda con demasiada información.

Capa 2 - Embeddings y VectorStore (ChromaDB)

Los fragmentos se convierten en vectores numéricos usando el modelo de embeddings paraphrase-multilingual-MiniLM-L12-v2 de Sentence Transformers. Inicialmente usé el modelo all-MiniLM-L6-v2 que recomendaba el enunciado, pero al probarlo con preguntas en español descubrí que el retriever fallaba a menudo: no encontraba fragmentos relevantes aunque existieran. Al cambiar a la variante multilingüe la mejora fue muy notable, ya que este modelo fue entrenado específicamente para más de 50 idiomas incluido el español. Los vectores se almacenan en ChromaDB persistido en disco, dentro de la carpeta propia de cada proyecto.

Capa 3 - Retriever (MMR)

En lugar de usar la búsqueda por similitud simple que sugería el enunciado, implementé MMR (Maximal Marginal Relevance). La diferencia es que MMR no solo busca los fragmentos más parecidos a la pregunta, sino que también penaliza la redundancia entre ellos. Así evito que los 5 resultados recuperados ($k=5$) sean prácticamente idénticos, lo que ocurría a menudo con las búsquedas simples cuando un texto repetía la misma idea en varios párrafos. Adicionalmente, configuré un parámetro `fetch_k=10`, que hace que ChromaDB recupere 10 candidatos internamente y luego seleccione los 5 más diversos de entre ellos.

También implementé un filtro de metadatos por archivo. Cuando el agente identifica a qué documento se refiere la pregunta del usuario, puede pasarle al retriever el nombre exacto del archivo para que solo busque en los fragmentos de ese documento. Esto evita lo que llamo contaminación cruzada de contexto: por ejemplo, si pregunto por vacaciones no quiero que me devuelva fragmentos de ciberseguridad que casualmente contengan la palabra "días".

Capa 4 - Cadena RAG con History-Aware Retriever (`rag_chain.py`)

La cadena RAG es la pieza central. En lugar de usar un RetrievalQA simple como sugería el enunciado, he implementado un pipeline LCEL (LangChain Expression Language) con dos etapas encadenadas:

- History-Aware Retriever: antes de buscar en la base vectorial, un primer prompt analiza el historial de chat para reformular la pregunta actual en una consulta independiente. Por ejemplo, si primero pregunto "¿Cuántos días de vacaciones tengo?" y luego digo "¿Y si llevo 7 años?", el sistema reformula la segunda pregunta a "¿Cuántos días extra de vacaciones corresponden con 7 años de antigüedad?" antes de buscar. Sin esto, la segunda pregunta por sí sola no tiene sentido para el buscador vectorial.
- Stuff Documents Chain: los fragmentos recuperados se inyectan en un prompt de Q&A personalizado que instruye al modelo para responder basándose estrictamente en el contexto, aplicar razonamiento lógico-matemático cuando hay tablas o rangos, y no mezclar normas de documentos diferentes.

Capa 5 - Agente + Tools (`agent.py`)

El agente se construye con `create_openai_tools_agent` y se ejecuta dentro de un `AgentExecutor`. He implementado tres herramientas decoradas con `@tool`:

- `consultar_documentos`: es la herramienta principal que ejecuta toda la cadena RAG descrita arriba. Recibe como parámetro la consulta y, opcionalmente, el nombre de un archivo para filtrar la búsqueda.
- `obtener_fecha_actual`: devuelve la fecha y hora del sistema. Útil para preguntas temporales.
- `calcular_estadisticas_texto`: calcula el número de palabras y caracteres de un texto proporcionado.

El agente utiliza un prompt con estrategia Few-Shot: en lugar de darle reglas largas y abstractas, le muestro ejemplos concretos de cómo debe invocar las herramientas. Tras varias iteraciones, descubrí que los modelos locales de 8B responden mucho mejor a ejemplos que a instrucciones genéricas.

Componentes adicionales que superan el enunciado

- API REST con FastAPI (`app/main.py`): toda la comunicación entre el frontend y la lógica RAG pasa por endpoints REST, lo que hace el sistema más profesional y escalable.
- Interfaz web con Streamlit (`frontend/app_ui.py`): un panel visual con sidebar para gestionar proyectos, subir documentos y chatear. Sustituye al bucle `input()` del enunciado.
- Memoria conversacional persistida (`ConversationSummaryBufferMemory`): el historial del chat se guarda físicamente en un archivo JSON por proyecto. Si el servidor se reinicia, la conversación se recupera intacta. Además, la memoria se autocompacta con un resumen cuando supera el límite de tokens, evitando desbordar la ventana de contexto del modelo.

- Multi-proyecto con aislamiento total: cada proyecto tiene su propio UUID, su propia carpeta de documentos, su propia base vectorial ChromaDB y su propio historial. No hay fugas de datos entre proyectos.

3. Justificación del modelo y parámetros

Modelo LLM

He trabajado con modelos de la familia meta-llama-3.1-8b-instruct cargados en LM Studio. Elegí este modelo porque es uno de los indicados en el enunciado y, tras probarlo, confirmé que ofrece un buen equilibrio entre velocidad de inferencia y calidad de respuesta para un equipo con 32 GB de RAM. Con temperature=0.0, las respuestas son deterministas y no son creativas, que es exactamente lo que necesito en un sistema RAG donde la fidelidad al documento es lo prioritario.

Modelo de Embeddings: paraphrase-multilingual-MiniLM-L12-v2

El enunciado sugería all-MiniLM-L6-v2, un modelo ligero pero entrenado fundamentalmente en inglés. Al probarlo con mis documentos en español, el retriever devolvía fragmentos poco relevantes o directamente no encontraba nada útil. Investigué alternativas y encontré la variante multilingüe del mismo modelo (paraphrase-multilingual-MiniLM-L12-v2), que está entrenado en más de 50 idiomas. El cambio supuso una mejora drástica en la precisión de la recuperación. Se descarga automáticamente la primera vez y ocupa más que el original, pero el beneficio compensa con creces.

chunk_size = 500, chunk_overlap = 50

Probé tres valores de chunk_size: 300, 500 y 800. Con 300 los fragmentos eran demasiado cortos y a menudo cortaban una idea a mitad de frase, lo que provocaba respuestas incompletas. Con 800 los fragmentos incluían demasiada información mezclada y el modelo tendía a confundir conceptos. Con 500 obtuve el mejor resultado: cada fragmento suele corresponder a un párrafo completo o a una sección bien delimitada de la política. El overlap de 50 caracteres garantiza que, si un concepto clave cae justo en el borde entre dos fragmentos, aparezca en ambos y no se pierda.

k = 5, fetch_k = 10 (con MMR)

Empecé con k=4 como decía el enunciado, pero al activar MMR subí a k=5 y fetch_k=10 para dar más margen al algoritmo de diversidad. Con k=4 y búsqueda simple me ocurría frecuentemente que los cuatro resultados eran variaciones del mismo párrafo. Con MMR y estos valores, los cinco fragmentos devueltos cubren diferentes aspectos de la pregunta, lo que mejora notablemente las respuestas en preguntas complejas.

temperature = 0.0

En un sistema RAG la prioridad es que el modelo se ciña al contexto recuperado y no invente. Con temperature=0.0 las respuestas son deterministas: la misma pregunta con el mismo contexto produce siempre la misma respuesta, lo que es ideal cuando estamos consultando políticas de empresa donde la exactitud es crítica. Probé con 0.3 y las respuestas eran algo más fluidas, pero a veces añadía matices o consejos que no estaban en los documentos.

4. Resultados

A continuación presento cinco ejemplos de preguntas realizadas al sistema junto con las respuestas obtenidas y mi evaluación de su calidad. Todos los ejemplos han sido ejecutados contra los cuatro documentos de Acme Corp cargados en un único proyecto.

Ejemplo 1

Pregunta:

«Llevo trabajando en Acme Corp 7 años. ¿Cuántos días extra de vacaciones me corresponden por antigüedad?»

Respuesta del sistema:

Con 7 años de antigüedad, te encuentras en el tramo de 5 a 10 años, por lo que te corresponden +3 días laborables extra. Los días devengados en un año natural deben disfrutarse antes del 31 de marzo del año siguiente.

Fuentes: Política vacaciones.txt

Evaluación:

Correcta. El sistema identifica que 7 años cae en el tramo 5-10 de la tabla de antigüedad y extrae el valor exacto (+3 días) sin copiar la tabla entera. Además añade un detalle relevante sobre el plazo de disfrute que encontró en el mismo documento.

Ejemplo 2

Pregunta:

«Me he comprado una silla ergonómica de 300 euros para trabajar desde casa. ¿Cuánto me cubren?»

Respuesta del sistema:

La subvención de Ergonomía te permite obtener hasta 250,00 euros para la compra de una silla ergonómica o monitor, siempre que acredites la factura correspondiente.

Fuentes: Política teletrabajo.txt

Evaluación:

Correcta. Localiza la regla de subvención en la política de teletrabajo, indica el tope máximo (250 euros) y la condición de presentar factura. Busca usando los sustantivos exactos del usuario ("silla ergonómica") sin inventar sinónimos.

Ejemplo 3

Pregunta:

«¿Cuál es la principal diferencia entre las sanciones leves y graves en ciberseguridad?»

Respuesta del sistema:

Según el contexto, las diferencias entre sanciones en ciberseguridad son:

- Leve: Dejar el equipo desbloqueado en zona pública. Sanción: Amonestación y curso de re-capacitación.
- Grave: Desactivar el antivirus o software de monitoreo. Sanción: Suspensión de 5 días de empleo y sueldo.
- Muy Grave: Extracción de bases de datos o pérdida de equipo no cifrado. Sanción: Despido disciplinario y acciones legales.

Fuentes: Ciberseguridad.txt

Evaluación:

Correcta y completa. El sistema no solo distingue entre leves y graves como pedía la pregunta, sino que además incluye la categoría "Muy Grave" con sus consecuencias. Cada nivel lleva su ejemplo concreto extraído del documento.

Ejemplo 4

Pregunta:

«¿Cuánto es el límite de alojamiento por noche en un viaje de trabajo?»

Respuesta del sistema:

El límite máximo para gastos de alojamiento es de 150,00 euros/noche, que incluye desayuno. La reserva obligatoria debe realizarse vía AcmeTravel.

Fuentes: Política gastos de viaje.txt

Evaluación:

Correcta. Extrae el límite exacto del documento (150 euros/noche) junto con dos condiciones relevantes: que incluye desayuno y que la reserva debe hacerse por el canal corporativo AcmeTravel.

Ejemplo 5

Pregunta:

«¿Qué opinas del nuevo iPhone 16?»

Respuesta del sistema:

No puedo emitir opiniones sobre productos o servicios. ¿Hay algo más en lo que pueda ayudarte?

Evaluación:

Correcta y esperada. El sistema reconoce que la pregunta no está relacionada con ningún documento indexado y rechaza responder sin alucinar ni inventar datos. Este comportamiento cumple el requisito del enunciado de no generar respuestas cuando la información no está en los documentos.

5. Limitaciones y mejoras

Limitaciones detectadas

- Rendimiento del modelo local: al ejecutarse en CPU (o GPU limitada), la latencia de cada respuesta es de varios segundos. En preguntas complejas donde el agente invoca la herramienta varias veces, la espera puede superar los 30 segundos. Esto es aceptable para un prototipo pero no para un entorno de producción con muchos usuarios simultáneos.
- Sensibilidad del prompt: los modelos locales de 8B parámetros son muy sensibles a la formulación del prompt del agente. Durante el desarrollo tuve que iterar varias veces entre estrategias de Zero-Shot (reglas) y Few-Shot (ejemplos) porque el modelo se sobrecargaba con instrucciones largas y alucinaba herramientas que no existían. La estrategia Few-Shot resultó más estable, pero requiere adaptar los ejemplos cada vez que cambian los documentos de la empresa. Por lo que finalmente mi intención de hacer un RAG multiproyecto, queda un poco inservible.
- Límites del razonamiento matemático: aunque he mejorado bastante el prompt, en ocasiones el modelo local sigue cometiendo errores aritméticos, especialmente cuando debe encadenar varias operaciones. Por ejemplo, calcular días devengados, sumarle bonificaciones por antigüedad y restarle días ya consumidos todo en una sola respuesta. Un modelo más grande (13B+) probablemente manejaría mejor estas cadenas de razonamiento.
- Búsqueda multi-documento limitada: aunque el agente puede realizar varias búsquedas secuenciales cambiando de archivo, en la práctica no siempre lo hace espontáneamente. A veces se conforma con la primera respuesta sin explorar otros documentos que podrían aportar información complementaria a la pregunta.

Mejoras con más tiempo

- Interfaz con streaming: actualmente el usuario ve un spinner mientras espera. Sería mejor implementar streaming token a token usando Server-Sent Events para que la respuesta aparezca progresivamente como en ChatGPT.
- Re-ranking: añadir una segunda etapa de reordenación de resultados con un modelo cross-encoder después del retriever. Esto mejoraría aún más la relevancia de los fragmentos seleccionados.
- Evaluación automatizada: crear un conjunto de preguntas con respuestas esperadas para medir automáticamente la precisión del sistema (tipo benchmark RAGAS). Así podría comparar objetivamente distintas configuraciones de chunk_size, k y modelos.
- Autenticación y roles: en un despliegue real, cada usuario debería autenticarse y solo ver los proyectos que le corresponden. Actualmente cualquiera que acceda a la URL puede ver todos los proyectos.
- Soporte para más formatos: ampliar la ingesta a documentos Word (.docx), hojas de cálculo (.xlsx) y presentaciones (.pptx), que son formatos muy habituales en entornos empresariales.
- Investigar más sobre la implementación de sistemas multi-agente (router/supervisor) para acercarme a un sistema empresarial, que sea capaz de gestionar tareas complejas invocando agentes especializados en cada tarea en

lugar de intentar hacerlo todo en un solo agente.